PROJECT TITLE

OOP Analysis of Streamlit Library

Submitted By:

1. **Abdul Qudoos — 2046**
2. **Ahmed Hassan — 2062**
3. **Ali Hassan — 2065**

University:

The Islamia University of Bahawalpur

Course Name:

Object-Oriented Programming (BS Data Science – 2nd S - M-2)

Teacher Name:

Sir Akmal Shahbaz

Submission Date:

30 May 2026

1. Library Overview

What is Streamlit?

Streamlit is a Python library that's free for anyone to use.

It allows developers to build interactive web applications using pure Python, without needing HTML, CSS, or JavaScript.

It helps people who write code make web pages that can be used by people who do data science and machine learning work.

The people who made Streamlit are Adrien Treuille, Amanda Kelly and Thiago Teixeira.

They made Streamlit available to everyone for the time in October 2019.

Now Streamlit is taken care of by Snowflake Inc. Because they bought it in 2022.

Who Uses Streamlit?

i. Streamlit is used by people who do data science work machine learning work and research.
ii. These people use Streamlit because they want to make web pages that they can share with people.
iii. They take the code they write in Python.
iv. Turn it into web pages using Streamlit.
v. Big companies like Uber and Airbnb use Streamlit to make tools that they use inside their companies.
vi. Streamlit is also used by people who do research to show people the models they make using machine learning.
vii. Streamlit is a tool that helps people who do data science and machine learning work and companies, like Uber and Airbnb use Streamlit for this purpose.

Installation:

pip install streamlit

To install Streamlit, run this command.

streamlit run filename.py

For output Streamlit code, run this command.

Why we picked Streamlit for OOP Analysis?

Streamlits internal setup is an example of Object-Oriented Design.

- We looked at its core class **DeltaGenerator**. It uses inheritance through **Mixin** classes.
- **SessionState** class in Streamlit shows encapsulation.
- The **st.write()** function in Streamlit is an example of polymorphism.

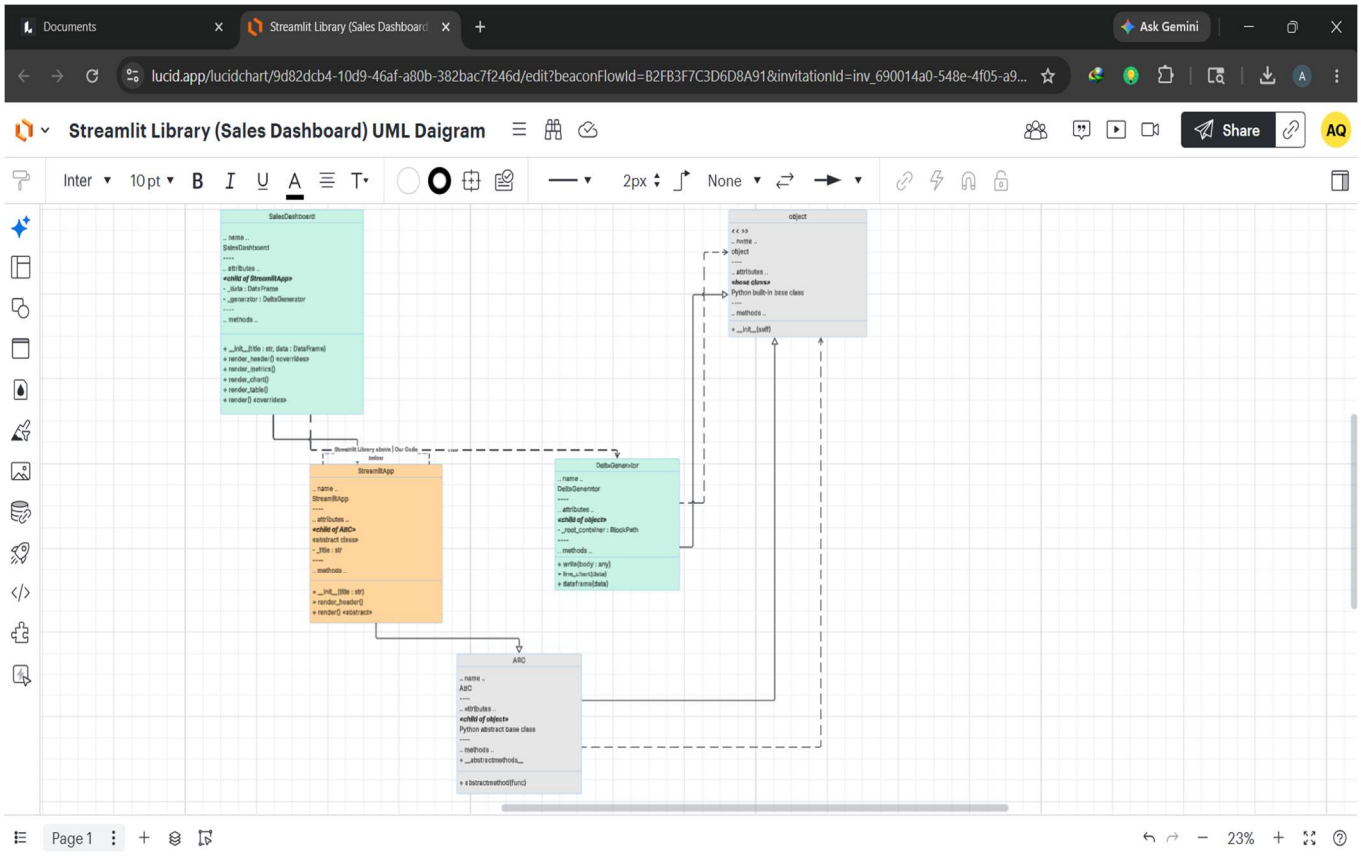
All these features make Streamlit a perfect real-world example for studying OOP principles.

Streamlit is great, for learning about Object-Oriented Design and Object-Oriented Programming.

That is why we choose Streamlit for **Object-Oriented Programming analysis**.

2. Class Hierarchy Diagram

The diagram below illustrates the complete class hierarchy of our project. The top of the hierarchy begins with Python's built-in object class, followed by Streamlit's internal classes, and finally our own custom classes at the bottom.



2.2 Description of Each Class

Class Name	Type	Role in Project
object	Python Built-in	Root base class — every class in Python inherits from this
ABC	Python Built-in	Abstract Base Class — forces child classes to implement abstract methods
DeltaGenerator	Streamlit Library	Core Streamlit class — provides all st.write(), st.button(), st.line_chart() methods
StreamlitApp	Our Custom Class	Abstract parent class — defines the template that all dashboards must follow
SalesDashboard	Our Custom Class	Child class — inherits StreamlitApp and uses DeltaGenerator to build the full dashb

2.3 Relationship Types Used

Relationship	Style	Example in our project
Inheritance	Solid line + hollow triangle arrow	StreamlitApp inherits from ABC
Inheritance	Solid line + hollow triangle arrow	SalesDashboard inherits from StreamlitApp
Uses / Composition	Dashed line + open arrow	SalesDashboard uses DeltaGenerator via self._generator

3. OOP Principles Analysis

1. Encapsulation:

Encapsulation means hiding internal data inside an object and only exposing what is necessary through public methods. In our project, all internal variables use a single underscore prefix to mark them as private, preventing direct access from outside the class.

Example from our code — StreamlitApp and SalesDashboard:

```
class StreamlitApp(ABC):
    def __init__(self, title: str):
        self._title = title    # private – cannot be changed directly from outside

class SalesDashboard(StreamlitApp):
    def __init__(self, title, data):
        super().__init__(title)
        self._data      = data          # private – only class methods can use this
        self._generator = st._main      # private – DeltaGenerator instance
```

The underscore prefix (_title, _data, _generator) signals that these variables belong strictly to the class. External code cannot modify them directly, which protects the internal state of the object.

2. Inheritance:

Inheritance allows a child class to automatically receive all the attributes and methods of its parent class without rewriting them. In our project, SalesDashboard inherits from StreamlitApp, which itself inherits from ABC (Abstract Base Class).

Example from our code — inheritance chain:

```
# StreamlitApp inherits from ABC (Python built-in)
class StreamlitApp(ABC):
    def render_header(self):   # this method is inherited by all child classes
        st.title(self._title)

# SalesDashboard inherits from StreamlitApp
class SalesDashboard(StreamlitApp):
    def __init__(self, title, data):
        super().__init__(title)   # calling parent constructor

# Now SalesDashboard automatically has render_header() — no need to rewrite it
app = SalesDashboard('My Dashboard', data)
app.render_header()   # works because it is inherited from StreamlitApp
```

The super().__init__(title) call ensures the parent class constructor runs first, correctly setting up the _title attribute before SalesDashboard adds its own variables.

3. Polymorphism:

Polymorphism means the same method name behaves differently depending on which class it belongs to. In our project, the render() and render_header() methods are defined in StreamlitApp but overridden in SalesDashboard with completely different behavior.

Example from our code — method overriding:

```
# In StreamlitApp (parent) — simple version
def render_header(self):
    st.title(self._title)    # only shows the title

# In SalesDashboard (child) — overridden version
def render_header(self):
    super().render_header()                              # runs parent version first
    st.caption('The Islamia University of Bahawalpur')  # then adds more content
    st.markdown('---')

# Same method name — completely different behavior
# This is Polymorphism
```

Streamlit's own st.write() method is another powerful example of polymorphism — it accepts a string, a number, a DataFrame, or a dictionary and renders each one appropriately using a single function call.

4. Abstraction:

Abstraction means hiding complex implementation details and exposing only a simple interface to the user. In our project, we used Python's ABC module to create an abstract class that forces all child classes to implement the render() method.

Example from our code — abstract method:

```
from abc import ABC, abstractmethod

class StreamlitApp(ABC):           # ABC makes this an abstract class
    @abstractmethod
    def render(self):              # abstract method — intentionally left empty
        pass                       # child class MUST implement this

# SalesDashboard is forced to implement render()
class SalesDashboard(StreamlitApp):
    def render(self):              # full implementation hidden inside
        self.render_header()
        self.render_metrics()
        self.render_chart()
        self.render_table()

# The user just calls one line — all complexity is hidden
app.render()    # Abstraction in action
```

From the user's perspective, app.render() is a single simple call. The internal complexity — setting up columns, drawing charts, formatting tables — is completely hidden inside the class methods. This is the essence of abstraction.

4. Design Decision Critique

The Decision: Functional API vs Object-Oriented API:

Streamlit made a deliberate architectural decision to expose its functionality through a functional API (st.write(), st.button()) rather than a class-based, object-oriented API. This is a significant design choice that has both advantages and disadvantages.

Why Streamlit Chose a Functional API?

Streamlit's functional approach:

```
import streamlit as st

# Just call functions — no class, no object, no setup
st.title('My App')
st.write('Hello World')
st.line_chart(data)
```

This approach was chosen to make Streamlit as beginner-friendly as possible. A data scientist can turn a Python script into a web app by simply adding st. before their function calls. No knowledge of web development or OOP is required

Trade-offs of This Decision!

Advantage	Disadvantage
Very easy to learn — no OOP knowledge needed	Hard to reuse components across projects
Scripts look natural — just add st. prefix	Testing is difficult — no class to mock
Fast prototyping	Large apps become messy and unorganized
Small learning curve for beginners	No proper encapsulation of page state

Alternative: Class-Based API (like Dash)

```
# Dash uses a proper class-based, OOP-friendly API
import dash
from dash import html

app = dash.Dash(__name__)          # proper object creation
app.layout = html.Div([            # layout is a class attribute
    html.H1('My Dashboard'),
    html.P('Hello World')
])
app.run_server(debug=True)         # method call on the object
```

Dash's class-based approach is more aligned with proper OOP principles. It offers better testability, reusability, and separation of concerns. However, it has a steeper learning curve and requires more boilerplate code than Streamlit.

5. Custom Extension — Code

What We Built?

We built a Sales Dashboard application by extending Streamlit's core DeltaGenerator class through our custom class hierarchy. Our two classes — StreamlitApp (abstract parent) and SalesDashboard (child) — demonstrate all four OOP principles in a real working application.

Class Architecture:

Class	Inherits From	New Methods Added	OOP Concepts
StreamlitApp	ABC	render_header(), render() (abstract)	Abstraction, Encapsulation
SalesDashboard	StreamlitApp	render_metrics(), render_chart(), render_table(), render()	Inheritance, Polymorphism

Full Code — code.py:

```python
import streamlit as st
import pandas as pd
from abc import ABC, abstractmethod
from streamlit.delta_generator import DeltaGenerator

# BASE CLASS — StreamlitApp
class StreamlitApp(ABC):
    def __init__(self, title: str):
        self._title = title              # ENCAPSULATION
    def render_header(self):
        st.title(self._title)
    @abstractmethod
    def render(self):                    # ABSTRACTION
        pass

# CHILD CLASS — SalesDashboard
class SalesDashboard(StreamlitApp):      # INHERITANCE
    def __init__(self, title, data):
        super().__init__(title)
        self._data = data                # ENCAPSULATION
        self._generator: DeltaGenerator = st._main
    def render_header(self):             # POLYMORPHISM — overrides parent
        super().render_header()
        st.caption('IUB — BS Data Science')
        st.markdown('---')
    def render_metrics(self):
        col1, col2, col3 = st.columns(3)
        col1.metric('Total Sales',  f"${self._data['sales'].sum():,.0f}")
        col2.metric('Average Sale', f"${self._data['sales'].mean():,.0f}")
        col3.metric('Highest Sale', f"${self._data['sales'].max():,.0f}")
    def render_chart(self):
        st.subheader('Monthly Sales Chart')
        self._generator.line_chart(self._data)   # uses DeltaGenerator directly
    def render_table(self):
        st.subheader('Sales Data Table')
        self._generator.dataframe(self._data, use_container_width=True)
    def render(self):                    # POLYMORPHISM — overrides abstract method
        self.render_header()
        self.render_metrics()
        self.render_chart()
        self.render_table()

if __name__ == '__main__':
    data = pd.DataFrame({
        'sales': [150,230,180,290,310,270,380,410,360,490,520,480],
        'month': ['Jan','Feb','Mar','Apr','May','Jun',
                  'Jul','Aug','Sep','Oct','Nov','Dec']
    }).set_index('month')
    app = SalesDashboard('Monthly Sales Dashboard', data)
    app.render()
```

Run command:

```
streamlit run code.py
```

6. Comparison with Alternative Library

Streamlit vs Dash (by Plotly):

Feature	Streamlit	Dash (by Plotly)
API Style	Functional (st.write())	Object-Oriented (app = Dash())
OOP Friendliness	Low — no classes exposed to user	High — explicit app object
Ease of Learning	Very easy — beginner friendly	Moderate — requires OOP knowledge
Testability	Difficult — no class to mock	Easy — class-based, mockable
Reusability	Low — hard to reuse components	High — components as classes
Boilerplate Code	Minimal	More setup required
Best For	Quick prototypes, ML demos	Enterprise, production dashboards
Multiple Inheritance	Used internally (Mixin classes)	Not prominently used
Encapsulation	Internal only — not for users	Available through app class

Conclusion:

From an OOP perspective, Dash provides a more structured, class-based architecture that is better suited for large-scale enterprise applications. Streamlit, on the other hand, prioritizes developer experience and speed of development. For our project, we chose Streamlit because its internal OOP architecture — particularly its use of multiple

inheritance through Mixin classes — provides an excellent case study for analyzing real-world OOP design patterns.

7. References:

1. Streamlit Official Documentation — https://docs.streamlit.io
2. Streamlit GitHub Repository — https://github.com/streamlit/streamlit
3. Python ABC Module — https://docs.python.org/3/library/abc.html
4. Streamlit DeltaGenerator Source Code — https://github.com/streamlit/streamlit/blob/develop/lib/streamlit/delta_generator.py
5. Dash by Plotly Documentation — https://dash.plotly.com
6. Python OOP Documentation — https://docs.python.org/3/tutorial/classes.html
7. Snowflake Acquires Streamlit (2022) — https://www.snowflake.com/blog/snowflake-to-acquire-streamlit